CPE 334 — Project Deliverable 1

Engineering baseline and working Sprint 1 · Semester 1/2026 · Due 8 November 2026

Students	**vieng** (GitHub `vienggg`) — **full name, student ID** **ninja** (GitHub `NinjoMUDA`) — **full name, student ID**
Theme / context	Theme 4 E-Commerce / Ordering — **Campus Café Pre-order**: customers of one campus café order ahead; café staff (operators) accept orders in arrival order
Repository	https://github.com/vienggg/Food-order
Submitted release	Commit `85735ee` (branch `docs/deliverable-1-pdf`). Release tag on `main`: **filled in when the release is merged**
Results on that commit	Server unit + API 63/63, client unit 5/5, end-to-end 20/20 (phone 375 px + desktop 1280 px), build OK, 46/46 Sprint 1 acceptance criteria Pass, 0 failed, 0 skipped

Navigation

Part	Pages	Relevant IDs
Part A — SRS and product scope	2–5	Actors, scope, FR-01..FR-18, BR-01..BR-07, NFR-01..NFR-09, AS-01..06, DEC-01..08
Part B — Feature inventory and traceability	6–10	F-01..F-12; FR/BR/NFR → feature → sprint → AC → TEST → result
Part C — System-Level SDS	11–18	Architecture, data model, API conventions, role matrix (5.2), sessions, configuration, GCP plan
Part D — Three-sprint plan	19–20	Sprint goals, F-01..F-12 allocation, dependencies, Sprint 1 scope, risks
Part E — Sprint 1 engineering contracts	21–38	F-01..F-04; AC-F01-01..AC-F04-13 (46 AC); TEST-F01-01..TEST-F04-13 (58 tests); Definition of Done
Part F — Working Sprint 1 and verification	39	Demonstrations, build/test results, AC → test results, test excerpts, TDD, Issues/PRs, board, contribution, limitations

Contents

Part	Section	Page
A	Software Requirements Specification — Campus Café Pre-order	2
B	Feature inventory — Campus Café Pre-order	6
B	Traceability matrix — Campus Café Pre-order	8
C	System-Level Software Design Specification — Campus Café Pre-order	11
D	Three-sprint plan — Campus Café Pre-order	19
E	F-01 Login, logout, current user — engineering contract	21
E	F-02 Browse menu — feature contract	25
E	F-03 Place order, my orders, order detail — feature contract	28
E	F-04 Operator queue + accept order — engineering contract	34
E	Product Definition of Done — Sprint 1	38
F	docs/evidence/part-f.md	39

Part A — SRS and product scope

Software Requirements Specification — Campus Café Pre-order

Deliverable 1, part A. Issue #6. Owner: A (vieng). Source of decisions: `PLAN.md`. Requirement IDs are stable and match `traceability.md` and `features.md`. Version 1 — 6 Nov 2026. Later changes go in the change log at the bottom.

1. Problem and context

Problem. At lunch time students and staff queue at the campus café, order at the counter and then wait again while the food is made. The café cannot see demand before people arrive, and customers waste their short break waiting.

Context. One campus café (one shop, one menu, one queue). Customers order on their phone or laptop before they arrive and pick up when it is ready. Café staff see incoming orders in one shared queue and accept them in arrival order.

Goal. Customers order ahead and know the status of their order; staff see orders before customers arrive and work through them in order.

2. Actors and responsibilities

Actor	Who	Responsibilities in the system
Customer (`CUSTOMER`)	Student or university staff member with a seeded account	Browse the menu, place orders, see own orders and their status; later cancel before acceptance and pay (simulated)
Operator (`OPERATOR`)	Café staff	See all new orders (one shop: every operator sees every order), accept them; later reject with a reason, move orders through preparation to pick-up, manage the menu and sold-out items, see sales
Anonymous visitor	Anyone not logged in	Can only open the login page
System	The server	Enforces login, roles, ownership and all business rules; calculates totals; keeps an order's lines unchanged after it is placed

3. Scope

In scope (whole product, 3 sprints)

- Login, logout and current-user identity with two roles (Sprint 1)
- Menu with prices and availability (Sprint 1); menu management and sold-out toggle (Sprint 2)
- Place an order, see own orders and order detail (Sprint 1)
- Operator order queue and accept (Sprint 1); reject, customer cancel, fulfilment PREPARING → READY → PICKED_UP (Sprint 2)
- Simulated payment status (Sprint 2)
- Pickup time slots with order cut-off, sales summary, order search/filter, hardening (Sprint 3)

Explicit exclusions

- Self-registration, password recovery, account administration (seeded accounts only; allowed by the course brief)
- Real payment providers or storing card data (payment is simulated)
- More than one shop, delivery, loyalty points, discounts or coupons
- Item options (size, sugar level) and stock counts: an item is either available or sold out
- Push notifications, email or SMS; the customer refreshes the page to see status
- Native mobile apps (the web app works on phone and desktop browsers)

4. Functional requirements

Each requirement is observable through the UI and/or API. Detailed endpoints and acceptance criteria are in the feature contracts (`docs/sprints/sprint-1/`).

ID	Requirement	Feature	Sprint
FR-01	**Log in.** A user submits email and password. If they match a seeded account, the system starts a session and the user lands on their role's home page (customer → menu, operator → order queue). If not, the system shows "Incorrect email or password." and starts no session.	F-01	1
FR-02	**Log out.** A logged-in user logs out; the session ends on the server and any later request with the old session is treated as not logged in.	F-01	1
FR-03	**Current user.** The app can ask "who am I" and gets the user's name and role, or "not logged in". The UI shows name and role in the header and sends the user to their role's home page.	F-01	1
FR-04	**Role check on the server.** Every protected endpoint checks the session: no session → 401, wrong role → 403. The role always comes from the session, never from the request.	F-01 (used by F-02..F-04)	1
FR-05	**View menu.** A customer sees every menu item with name and price in baht. Sold-out items are shown greyed out with "SOLD OUT" and cannot be added to the cart.	F-02	1
FR-06	**Place an order.** A customer sends a list of menu items and quantities. If all rules pass (BR-01..BR-04), the system saves the order as PLACED with its lines and a server-calculated total, and shows the order. If any rule fails, nothing is saved and the reason is shown.	F-03	1
FR-07	**List my orders.** A customer sees only their own orders, newest first, with order number, time, status and total.	F-03	1
FR-08	**My order detail.** A customer opens one of their orders and sees each line (name, unit price, quantity, line total), the total and the status. Another customer's order, or a missing id, shows "Order not found."	F-03	1
FR-09	**Operator queue.** An operator sees all PLACED orders, oldest first, with order number, customer name, time, items and total.	F-04	1
FR-10	**Accept an order.** An operator accepts a PLACED order; it becomes ACCEPTED and leaves the queue, and the customer sees ACCEPTED.	F-04	1
FR-11	**Menu management.** An operator adds and edits menu items (name, price) and marks an item sold out or available again.	F-05	2
FR-12	**Customer cancel.** A customer cancels their own order while it is still PLACED; it becomes CANCELLED.	F-06	2
FR-13	**Operator reject.** An operator rejects a PLACED order with a reason; it becomes REJECTED and the customer sees the reason.	F-06	2
FR-14	**Fulfilment.** An operator moves an order ACCEPTED → PREPARING → READY → PICKED_UP, one step at a time.	F-07	2
FR-15	**Simulated payment.** An order has a payment status UNPAID → PAID → REFUNDED, changed by a simulated payment step and by refunds of paid orders that are cancelled or rejected.	F-08	2
FR-16	**Pickup time slots.** A customer chooses a pickup slot instead of ASAP; orders for a slot close at a cut-off time before it.	F-09	3
FR-17	**Sales summary.** An operator sees order count and sales total per day and the best-selling items.	F-10	3
FR-18	**Search / filter.** Customers filter their orders by status; operators search orders by number, customer or status.	F-11	3

5. Business rules

Each rule is enforced by the server, whatever the client sends. Money is stored as whole satang (15500 = ฿155.00).

ID	Rule	When broken
BR-01	The quantity on each order line is a whole number from 1 to 10.	400 VALIDATION_ERROR; nothing saved
BR-02	An order has 1 to 20 lines, and the same menu item cannot appear on two lines.	400 VALIDATION_ERROR; nothing saved
BR-03	Every ordered item must exist and be available (not sold out) at the moment the order is placed. If any one item fails, the whole order is rejected and nothing is saved (one database transaction).	400 VALIDATION_ERROR naming the bad item(s); no order and no lines stored
BR-04	The server calculates the total from the database prices. A request that contains any price or total field is rejected.	400 VALIDATION_ERROR; nothing saved
BR-05	Each order line keeps a copy of the item name and unit price at the time the order was placed; later menu changes do not change existing orders.	— (checked by reading the order after a menu change)
BR-06	Only an order with status PLACED can be accepted.	409 INVALID_STATE; status unchanged
BR-07	If two operators accept the same order at the same moment, exactly one succeeds and the other is told it was already accepted.	One 200, one 409 INVALID_STATE; order accepted once

6. Non-functional requirements

ID	Requirement	Measurable target	How we verify	Sprint
NFR-01	**Credentials and sessions are protected.**	Passwords stored only as bcrypt hashes (cost ≥ 10). Session id is random (≥ 128 bits) in an httpOnly, SameSite=Lax cookie (Secure in production). Session expires 8 h after login; logout deletes it.	API tests: stored hash ≠ password; cookie flags; expired and logged-out sessions → 401	1
NFR-02	**The server is the authority.**	0 endpoints take the user id, role, price or total from the request. Another customer's order returns 404 with no order data. Error responses never contain stack traces.	API tests for 401/403/404 per protected endpoint; test that a price field → 400; test that a 500 has no stack	1
NFR-03	**One error format.**	100% of API errors use `{ error: { code, message, details } }` with VALIDATION_ERROR 400, UNAUTHENTICATED 401, FORBIDDEN 403, NOT_FOUND 404, INVALID_STATE 409.	API tests assert code and shape on every error path	1
NFR-04	**Data integrity.**	A rejected order leaves 0 rows. 20 parallel accepts of one order give exactly 1 success.	API/DB tests: row count after rejection; concurrency test	1
NFR-05	**Automated tests, tests first.**	Every Sprint 1 acceptance criterion maps to ≥ 1 automated test; all pass on the release commit; rule tests have a failing run saved before the code.	Test run output on the release commit; red/green commits and saved output in `docs/evidence/tdd/`	1
NFR-06	**Usable states on phone and desktop.**	Every Sprint 1 screen has loading, empty and error states as in `docs/ui/ui-spec.md`; works at 375 px and 1280 px wide without sideways scrolling.	End-to-end tests at both widths + screenshots	1
NFR-07	**Deployable and configurable.**	All settings (database URL, seed password, port) come from environment variables; 0 secrets in Git; app runs as one container (Express serves the built React app).	Review + `git grep` for secrets each release; container build in Lab 5	1 (design), 2 (Lab 5)
NFR-08	**Login rate limit.**	After 5 failed logins for one email within 15 minutes, further attempts get 429 for 15 minutes.	API test	3
NFR-09	**Accessibility.**	Every Sprint 1–3 screen has no critical or serious issues in an automated accessibility check (axe), and works by keyboard only.	Playwright + axe check; manual keyboard check	3

7. Assumptions

ID	Assumption
AS-01	There is one café and one shared queue; any operator may act on any order.
AS-02	Accounts are seeded; the course does not require registration or password recovery.
AS-03	Pickup is "as soon as possible" until pickup slots arrive in Sprint 3.
AS-04	Prices are in Thai baht and do not include separate tax or service charges.
AS-05	Customers refresh to see status changes; real-time updates are not needed.
AS-06	Users have a modern browser (latest Chrome, Edge, Safari or Firefox).

8. Decisions

ID	Decision	Why
DEC-01	Sessions in a database table with a random id in an httpOnly cookie, 8 h expiry (not JWT).	Logout really ends the session; the browser's script cannot read the cookie; simple to test.
DEC-02	Same error for unknown email and wrong password.	Attackers cannot find which emails have accounts.
DEC-03	Money as integer satang.	No rounding errors with decimals.
DEC-04	Availability is a yes/no flag per item, not a stock count.	Matches how the café works (an item runs out); smallest version that works.
DEC-05	Order statuses and allowed moves are kept in one transition table with `canTransition()`. Sprint 1 allows only PLACED → ACCEPTED.	Later sprints add rows instead of new code paths; easy to unit-test.
DEC-06	Accept uses a conditional update `where status = PLACED`.	Two operators cannot both accept (BR-07) without locks.
DEC-07	Another customer's order returns 404, not 403.	Does not reveal that the order exists.
DEC-08	Payment status is separate from order status.	Paying and preparing are independent; keeps the order lifecycle simple.

9. Questions and resolution

ID	Question	Resolution
Q-01	One café or many?	One café (AS-01).
Q-02	Do operators only see orders they accepted?	No, one shared queue; all operators see all orders.
Q-03	Choose a pickup time in Sprint 1?	No, ASAP in Sprint 1; slots in Sprint 3 (FR-16).
Q-04	What happens if an item sells out while it is in a cart?	The server rejects the whole order on submit (BR-03); the UI reloads the menu and marks the line.
Q-05	Can the client send the price to save a database lookup?	No, any price/total field is rejected (BR-04).
Q-06	403 or 404 for another customer's order?	404 (DEC-07).

Change log

Date	Version	Change	Why
6 Nov 2026	1	First version	Deliverable 1 part A

Part B — Feature inventory and traceability

Feature inventory — Campus Café Pre-order

Deliverable 1, part B (1 of 2). Issue #7. Traceability: `traceability.md`. Sprint plan: `sprints/plan.md`. Version 1 — 6 Nov 2026 (draft). Requirement IDs match the SRS (#6).

Roles: **CUSTOMER** (orders food), **OPERATOR** (café staff; one shop, sees all orders). Anonymous users can only reach `/login`.

Summary

ID	Feature	Sprint	Roles	Requirements	Business rules	Status
F-01	Login, logout, "who am I"	1	anyone → CUSTOMER / OPERATOR	FR-01..FR-04, NFR-01, NFR-02	—	Verified
F-02	Browse menu	1	CUSTOMER	FR-05	BR-03 (sold-out shown)	Verified
F-03	Place order, my orders, order detail	1	CUSTOMER	FR-06..FR-08, NFR-04	BR-01, BR-02, BR-03, BR-04, BR-05	Verified
F-04	Operator queue + accept order	1	OPERATOR	FR-09, FR-10, NFR-04	BR-06, BR-07	Verified
F-05	Menu management + sold-out toggle	2	OPERATOR	FR-11	to specify in Sprint 2	Not started
F-06	Customer cancel / operator reject with reason	2	CUSTOMER, OPERATOR	FR-12, FR-13	to specify in Sprint 2	Not started
F-07	Fulfilment: PREPARING → READY → PICKED_UP	2	OPERATOR	FR-14	to specify in Sprint 2	Not started
F-08	Simulated payment	2	CUSTOMER, OPERATOR	FR-15	to specify in Sprint 2	Not started
F-09	Pickup time slots + order cut-off	3	CUSTOMER, OPERATOR	FR-16	to specify in Sprint 3	Not started
F-10	Sales summary	3	OPERATOR	FR-17	to specify in Sprint 3	Not started
F-11	Order search / filter	3	CUSTOMER, OPERATOR	FR-18	to specify in Sprint 3	Not started
F-12	Hardening: login rate limit, error states, accessibility	3	all	NFR-08, NFR-09	to specify in Sprint 3	Not started

All Sprint 1 features also follow NFR-03, NFR-05, NFR-06 and NFR-07.

Status values: Not started → Planned (in a sprint plan) → Specified (contract committed) → Implemented → Verified (tests pass on the release) → Released (tagged on `main`).

Sprint 1 features

F-01 Login, logout, "who am I"

- **What:** a seeded user logs in with email + password; the server creates a session; the app shows the user's name and role and sends them to their home page. Logout ends the session.
- **Screens:** S1 Login, shared header and route guards.
- **API:** `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`.
- **Key decisions:** sessions in the `sessions` table, random id in an httpOnly cookie, 8 hours; bcrypt hashes; role always read from the session; same 401 message for unknown email and wrong password. No sign-up (seeded accounts

only).
- **Issue / owner:** #2 / A (vieng). Contract: `docs/sprints/sprint-1/F-01-*.md`.

F-02 Browse menu

- **What:** a customer sees every menu item with name and price; sold-out items are greyed out with a SOLD OUT badge and cannot be added.
- **Screens:** S2 Menu (menu list part).
- **API:** `GET /api/menu` → `{ id, name, price, isAvailable }`; price in satang.
- **Access:** CUSTOMER only (401 if not logged in, 403 for OPERATOR).
- **Issue / owner:** #3 / B (ninja). Contract: `docs/sprints/sprint-1/F-02-*.md`.

F-03 Place order, my orders, order detail

- **What:** a customer builds a cart and places an order; the server checks every rule, calculates the total from database prices, and saves the order and its lines in one transaction. The customer can list their own orders and open one.
- **Screens:** S2 Menu (cart part), S3 My orders, S4 Order detail.
- **API:** `POST /api/orders` `{ items: [{ menuItemId, quantity }] }`, `GET /api/orders`, `GET /api/orders/:id`.
- **Rules:** BR-01 quantity 1–10; BR-02 1–20 lines, no duplicate item; BR-03 every item exists and is available, else nothing is saved; BR-04 server total, a price/total field in the request → 400; BR-05 each line keeps the item name and unit price at order time.
- **Access:** CUSTOMER only; another customer's order → 404 (same as a missing id).
- **Issue / owner:** #4 / B (ninja). Contract: `docs/sprints/sprint-1/F-03-*.md`.

F-04 Operator queue + accept order

- **What:** an operator sees PLACED orders oldest first and accepts one; it becomes ACCEPTED and leaves the queue.
- **Screens:** S5 Operator order queue.
- **API:** `GET /api/operator/orders?status=PLACED`, `POST /api/operator/orders/:id/accept`.
- **Rules:** BR-06 only a PLACED order can be accepted (else 409); BR-07 two operators accepting at once → exactly one 200, the other 409 (conditional update `where status = PLACED`). Uses the order transition table and `canTransition()`.
- **Access:** OPERATOR only (403 for CUSTOMER).
- **Issue / owner:** #5 / A (vieng). Contract: `docs/sprints/sprint-1/F-04-*.md`.

Shared groundwork (not a feature)

- Issue #1 setup: database schema (`users`, `sessions`, `menu_items`, `orders`, `order_lines`), seed data (2 customers, 1 operator, menu with one sold-out item; password from env `SEED_PASSWORD`), project skeleton and test setup. Used by all Sprint 1 features.
- Error shape for every feature: `{ error: { code, message, details } }` with VALIDATION_ERROR 400, UNAUTHENTICATED 401, FORBIDDEN 403, NOT_FOUND 404, INVALID_STATE 409.

Change log

Date	Change	Why
6 Nov 2026	First draft	Deliverable 1 part B
6 Nov 2026	F-01..F-04 status Verified: all tests pass on `sprint-1` (see traceability.md) (#28)	AC-B3

Traceability matrix — Campus Café Pre-order

Deliverable 1, part B (2 of 2). Issue #7. Features: `features.md`. Requirements: `docs/srs.md` (#6). Version 1 — 6 Nov 2026 (draft).

How to read a row: requirement → feature → sprint → acceptance criteria (from the feature contract) → test IDs (from the test spec) → result on the tested commit. Sprint 1 AC and test IDs come from the contracts in `docs/sprints/sprint-1/` and match the test names (`it("TEST-…")`). **Result** = test run on commit `f3acfc9` (6 Nov 2026): `npm test` server 63/63 + client 5/5 passed, `npm run test:e2e` 20/20 passed (phone + desktop). Re-run on the final `sprint-1` commit before submission.

1. Business rules

Req	Rule (short)	Feature	Sprint	AC	Tests	Result
BR-01	Quantity per line is a whole number 1–10	F-03	1	AC-F03-02, AC-F03-12	TEST-F03-02, -03 (API); -18 (unit)	Pass
BR-02	1–20 lines, no duplicate menu item	F-03	1	AC-F03-03, AC-F03-12	TEST-F03-04, -05, -06, -07 (API); -17, -18 (unit)	Pass
BR-03	Every item exists and is available, else nothing is saved	F-02, F-03	1	AC-F02-02, AC-F02-06, AC-F03-04, AC-F03-11	TEST-F02-02 (API), -08 (E2E); TEST-F03-08, -09 (API), -20 (E2E)	Pass
BR-04	Server calculates the total; price/total in request → 400	F-03	1	AC-F03-01, AC-F03-05	TEST-F03-01, -10 (API)	Pass
BR-05	Order line keeps item name and unit price at order time	F-03	1	AC-F03-06	TEST-F03-11 (API)	Pass
BR-06	Only a PLACED order can be accepted, else 409	F-04	1	AC-F04-05, AC-F04-06, AC-F04-10	TEST-F04-05, -06 (API); -10 (unit)	Pass
BR-07	Two operators accept at once → one 200, one 409	F-04	1	AC-F04-07, AC-F04-13	TEST-F04-07 (API, 20 parallel); -13 (E2E)	Pass

2. Functional requirements

IDs agreed with the SRS author in chat on 6 Nov 2026.

Req	Requirement (short)	Feature	Sprint	AC	Tests	Result
FR-01	Log in with email + password	F-01	1	AC-F01-01..05, AC-F01-12, AC-F01-13	TEST-F01-01..05 (API); -12, -13 (E2E)	Pass
FR-02	Log out	F-01	1	AC-F01-09, AC-F01-12	TEST-F01-09 (API); -12 (E2E)	Pass
FR-03	Show current user and send to role home	F-01	1	AC-F01-06, AC-F01-07, AC-F01-12, AC-F01-14	TEST-F01-06, -07 (API); -12, -14 (E2E)	Pass
FR-04	Server checks role on every protected endpoint	F-01 (used by F-02..F-04)	1	AC-F01-10, AC-F01-14, AC-F02-03, AC-F02-04, AC-F03-09, AC-F04-09	TEST-F01-10 (API), -14 (E2E); TEST-F02-03..05; TEST-F03-15, -16; TEST-F04-09 (API)	Pass
FR-05	View menu with price and availability	F-02	1	AC-F02-01, -02, -05, -06, -07	TEST-F02-01, -02, -06 (API); -07 (unit); -08, -09 (E2E)	Pass
FR-06	Place an order	F-03	1	AC-F03-01..05, AC-F03-10..12	TEST-F03-01..10 (API); -17, -18 (unit); -19, -20 (E2E)	Pass
FR-07	List my orders	F-03	1	AC-F03-07, AC-F03-10	TEST-F03-12 (API); -19 (E2E)	Pass
FR-08	View my order detail	F-03	1	AC-F03-06, AC-F03-08, AC-F03-10	TEST-F03-11, -13, -14, -21 (API); -19 (E2E)	Pass

Req	Requirement (short)	Feature	Sprint	AC	Tests	Result
FR-09	Operator queue of PLACED orders, oldest first	F-04	1	AC-F04-01..03, AC-F04-11	TEST-F04-01..03 (API); -11 (E2E)	Pass
FR-10	Accept an order	F-04	1	AC-F04-04..08, AC-F04-12, AC-F04-13	TEST-F04-04..08 (API); -12, -13 (E2E)	Pass
FR-11	Menu management	F-05	2	Sprint 2 contract	—	—
FR-12	Customer cancel	F-06	2	Sprint 2 contract	—	—
FR-13	Operator reject with reason	F-06	2	Sprint 2 contract	—	—
FR-14	Fulfilment PREPARING → READY → PICKED_UP	F-07	2	Sprint 2 contract	—	—
FR-15	Simulated payment	F-08	2	Sprint 2 contract	—	—
FR-16	Pickup time slots	F-09	3	Sprint 3 contract	—	—
FR-17	Sales summary	F-10	3	Sprint 3 contract	—	—
FR-18	Order search / filter	F-11	3	Sprint 3 contract	—	—

3. Non-functional requirements

Req	Requirement (short)	Feature	Sprint	AC	How verified / tests	Result
NFR-01	bcrypt passwords, httpOnly session cookie, 8 h expiry	F-01	1	AC-F01-01, -08, -09, -11; setup AC-1.2	TEST-F01-01, -08, -09 (API), -11 (unit); seed.test.ts AC-1.2 (bcrypt hash)	Pass
NFR-02	Role and prices only from server; no stack traces	F-01, F-03	1	AC-F01-04, -06; AC-F03-05, -08; setup AC-1.5	TEST-F01-04, -06; TEST-F03-10, -14 (API); app.test.ts "no stack trace"	Pass
NFR-03	One error format, codes 400/401/403/404/409	F-01..F-04	1	setup AC-1.4, AC-1.5; every error AC	app.test.ts (AC-1.5); every API test asserts `error.code`	Pass
NFR-04	One transaction per order; safe concurrent accept	F-03, F-04	1	AC-F03-04, AC-F04-07	TEST-F03-03..10 (0 orders and 0 lines after rejection); TEST-F04-07 (20 parallel accepts)	Pass
NFR-05	Automated unit, API and end-to-end tests; tests first	F-01..F-04	1	all Sprint 1 AC	All tests above; red/green output in `docs/evidence/tdd/` (F-01..F-04)	Pass
NFR-06	Loading, empty and error states; phone and desktop	F-01..F-04	1	AC-F01-13, AC-F02-07, AC-F03-10, -11, AC-F04-11	E2E on phone 375 px and desktop 1280 px (20 runs); screenshots in `docs/evidence/screenshots/`	Pass
NFR-07	One container, settings from env, no secrets in Git	all	1	—	Manual: `server/.env.example` placeholders only; `git grep` for the seed password: no match (6 Nov 2026). Container in Lab 5	Pass
NFR-08	Login rate limit	F-12	3	Sprint 3 contract	—	—
NFR-09	Accessibility	F-12	3	Sprint 3 contract	—	—

4. Coverage check

- Every FR/BR/NFR maps to a feature: **yes** (FR-01..FR-18, BR-01..BR-07, NFR-01..NFR-09).
- Every Sprint 1 AC appears in at least one row above: **yes** — AC-F01-01..14, AC-F02-01..07, AC-F03-01..12, AC-F04-01..13 (46 AC).
- Every Sprint 1 AC has at least one automated test: **yes** — 58 test IDs (TEST-F01-01..14, TEST-F02-01..09, TEST-F03-01..21, TEST-F04-01..13), all found in test names in `server/tests/`, `client/src/lib/`, `e2e/`.
- Only manual check: NFR-07 (secrets and configuration), documented above.

Test files	Level	IDs
`server/tests/auth.test.ts`	API (+ unit TEST-F01-11)	TEST-F01-01..11
`server/tests/menu.test.ts`	API	TEST-F02-01..06
`server/tests/orders.test.ts`	API	TEST-F03-01..16, -21
`server/tests/operator.test.ts`, `transitions.test.ts`	API, unit	TEST-F04-01..09, -10
`server/tests/app.test.ts`, `seed.test.ts`	API, DB	setup AC-1.2..1.5 (#1)
`client/src/lib/money.test.ts`, `cart.test.ts`	unit	TEST-F02-07, TEST-F03-17..18
`e2e/auth.spec.ts`, `menu.spec.ts`, `orders.spec.ts`, `queue.spec.ts`	E2E	TEST-F01-12..14, TEST-F02-08..09, TEST-F03-19..20, TEST-F04-11..13

Change log

Date	Change	Why
6 Nov 2026	First draft (business rules mapped)	Deliverable 1 part B
6 Nov 2026	AC, test IDs and results for all Sprint 1 rows (#28)	AC-B3

Part C — System-Level SDS

System-Level Software Design Specification — Campus Café Pre-order

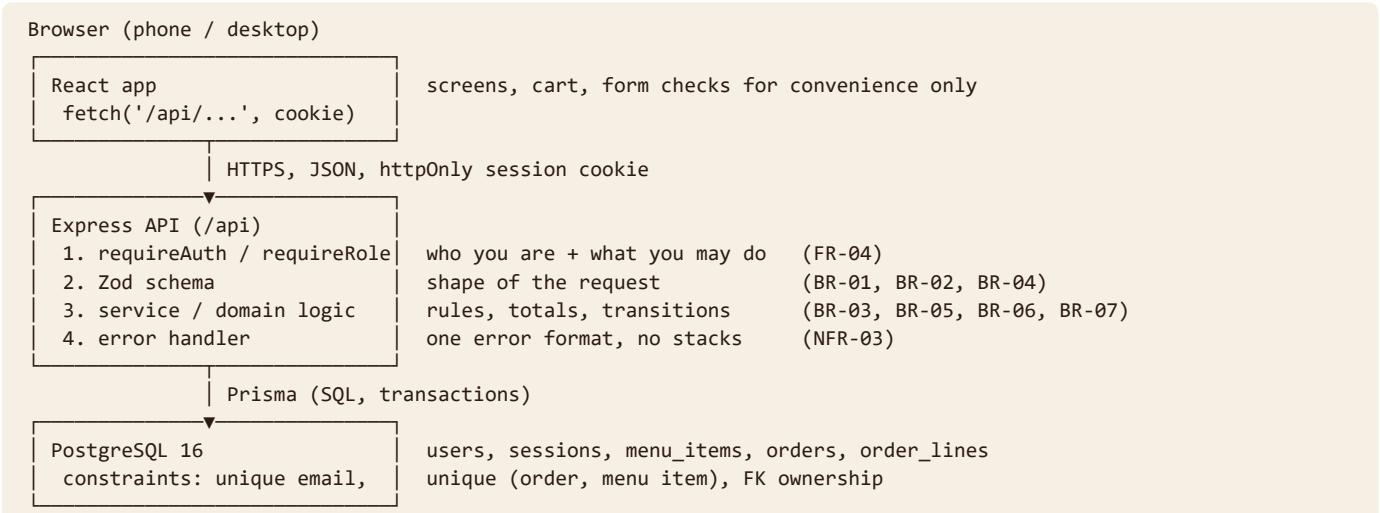
Deliverable 1, part C. Issue #8. Requirements: `srs.md`. Screens: `ui/ui-spec.md`. Authors: A (vieng) sections 1, 2, 5, 6, 7; B (ninja) sections 3, 4. Version 1 — 6 Nov 2026. Later changes go in the change log at the bottom.

1. Architecture

1.1 Stack

Layer	Technology	Why
UI	React 19 + Vite + TypeScript, React Router	Course stack; fast dev server; typed props
API / application	Node 24 + Express 5 + TypeScript, Zod for input checks	Course stack (Lab 5 kit); Zod gives strict request schemas (BR-04)
Database	PostgreSQL 16 + Prisma 6 (schema, migrations, seed)	Real transactions and row-level conditional updates (BR-03, BR-07)
Tests	Vitest + Supertest (unit + API against real Postgres), Playwright (end-to-end)	Tests run against the same database engine as production
Packaging	One Docker image: Express serves the built React app and `/api`	Lab 5 starter kit layout; one origin, so the session cookie needs no CORS

1.2 Components and responsibilities



Component	Does	Does not
React app	Shows screens and states; keeps the cart in memory; calls the API; shows `error.message` from the server	Decide prices, totals, roles or what a user may see — anything it checks is only for convenience
Express API	Authenticates every request from the session cookie; checks role and ownership; validates input; enforces all business rules; calculates totals; returns JSON	Trust any user id, role, price or total sent by the client
PostgreSQL	Stores data; guarantees uniqueness and foreign keys; runs each order in one transaction; makes "accept only if still PLACED" atomic	Hold business logic in triggers

1.3 Where rules are enforced

Rule	Enforced in	Backed up by
Logged in, role (FR-04)	`requireAuth` / `requireRole` middleware on every protected route	API tests for 401 and 403

Rule	Enforced in	Backed up by
Ownership (FR-08)	Order queries always filter by `customerId = session user`	API test: other customer's order → 404
BR-01, BR-02, BR-04	Strict Zod schema on `POST /api/orders`	DB unique `(order_id, menu_item_id)` for BR-02
BR-03	Order service inside one Prisma transaction	Nothing saved on error (transaction rolls back)
BR-05	Order service copies name and price into `order_lines`	Lines are never updated after insert
BR-06	`canTransition(from, to)` + transition table	Conditional update
BR-07	`updateMany where { id, status: PLACED }`; 0 rows updated → 409	Concurrency API test

1.4 Request flow example: place an order (F-03)

1. Browser sends `POST /api/orders` with `{ items: [{ menuItemId, quantity }] }` and the session cookie.
2. `requireRole('CUSTOMER')` loads the session; missing/expired → 401, operator → 403.
3. Zod strict schema checks BR-01, BR-02 and rejects any extra field such as `price` (BR-04) → 400.
4. In one transaction: load the items; any missing or sold out → 400, nothing saved (BR-03); compute the total from DB prices; insert the order and lines with copied name and price (BR-05).
5. Respond 201 with the saved order. The UI goes to `/orders/:id`.

1.5 Code layout

Path	Contents
`client/src/`	React screens, router, API helper
`server/src/app.ts`	Builds the Express app: JSON, cookies, routes, 404, error handler
`server/src/errors.ts`	`AppError` and the shared error handler
`server/src/db.ts`	The one Prisma client
`server/src/<feature>/`	Routes and logic per feature (added by F-01..F-04)
`server/prisma/`	`schema.prisma`, `migrations/`, `seed.ts`
`server/tests/`	Vitest + Supertest tests; `helpers.ts` resets the test DB
`e2e/`	Playwright tests

2. UI conventions

Full screen-by-screen states are in `ui/ui-spec.md`; final designs in `docs/ui/design/`.

2.1 Screens and navigation

Screen	Route	Role	Features
S1 Login	`/login`	public	F-01
S2 Menu + cart	`/menu` (customer home)	CUSTOMER	F-02, F-03
S3 My orders	`/orders`	CUSTOMER	F-03
S4 Order detail	`/orders/:id`	CUSTOMER	F-03
S5 Order queue	`/operator/orders` (operator home)	OPERATOR	F-04
Not found	any other path	any	—

`/` redirects: not logged in → `/login`, customer → `/menu`, operator → `/operator/orders`. **Route guard:** on start the app calls `GET /api/auth/me`. No session → `/login`. A route for the other role → own home with "You don't have access to that page." This only helps the user; the API still enforces every rule (section 5).

2.2 Shared behaviour

State	Behaviour
Loading	Grey placeholder rows; the action button is disabled and its label changes ("Logging in…", "Placing order…", "Accepting…"), which also stops double submits
Empty	Dashed box with one sentence and a next step where useful ("You haven't ordered yet." + Go to menu)
Validation (client)	Required/format checks and cart limits (qty 1–10, max 20 lines, sold-out items not addable). Convenience only; the server re-checks
Error from API	Show the server's `error.message`. 400 → error box, keep the user's input, mark fields/lines named in `details`. 401 → back to `/login` with "Your session has ended. Please log in again." 403 → own home with "You don't have access to that page." 404 → "not found" state with a back link. 409 → show message, reload the data
Network / 500	"Something went wrong. Please try again." + Try again
Money / time	Integer satang shown as `฿xx.xx`; times local (`10:15`, `Yesterday 12:02`, `3 Nov 09:40`)
Layout	Phone: bottom tab bar; desktop: sidebar. Works from 375 px to 1280 px+ (NFR-06)

3. Data model

Source of truth: `server/prisma/schema.prisma` and `server/prisma/migrations/` (Issue #1). Table and column names are snake_case in the database, camelCase in code.

3.1 Entities and relationships

```
users 1 ——— * sessions          (a user can be logged in on several devices)
users 1 ——— * orders            (customer_id; only CUSTOMER users place orders)
orders 1 —— * order_lines       (1–20 lines per order, BR-02)
menu_items 1 — * order_lines    (menu_item_id; name and price are copied, BR-05)
```

3.2 Tables

Table	Column	Type	Constraint / note
`users`	`id`	int	PK, auto
	`email`	text	**unique**; login looks it up case-insensitive
	`name`	text	shown in header and operator queue
	`password_hash`	text	bcrypt only, never plain text (NFR-01)
	`role`	enum `Role`	`CUSTOMER` | `OPERATOR`
	`created_at`	timestamp	default now
`sessions`	`id`	text	PK; 32 random bytes, base64url; the cookie value
	`user_id`	int	FK → `users.id`, **on delete cascade**; indexed
	`expires_at`	timestamp	login + 8 h; expired rows are refused and deleted
	`created_at`	timestamp	default now
`menu_items`	`id`	int	PK, auto
	`name`	text	**unique** (seed upserts by name)
	`price`	int	satang, > 0 (seed and F-05 validation)
	`is_available`	boolean	default true; false = sold out (DEC-04)
`orders`	`id`	int	PK, auto; shown to users as `#id`
	`customer_id`	int	FK → `users.id`; indexed; ownership check (FR-08)
	`status`	enum `OrderStatus`	default `PLACED`; values below
	`total`	int	satang; written once by the server (BR-04)

Table	Column	Type	Constraint / note
	`created_at`	timestamp	default now; queue and list ordering
	`updated_at`	timestamp	set by Prisma on every update
	`accepted_at`	timestamp, null	set by F-04 when accepted (migration in F-04)
`order_lines`	`id`	int	PK, auto
	`order_id`	int	FK → `orders.id`, on delete cascade
	`menu_item_id`	int	FK → `menu_items.id`
	`name`	text	copy of the menu name at order time (BR-05)
	`unit_price`	int	copy of the menu price at order time, satang (BR-05)
	`quantity`	int	1–10 (BR-01, checked by the API schema)
			unique (`order_id`, `menu_item_id`) — no duplicate item in one order (BR-02)

Indexes: `orders (status, created_at)` for the operator queue (PLACED, oldest first); `orders (customer_id)` for "my orders"; `sessions (user_id)`.

3.3 Order status

```
PLACED ──accept──► ACCEPTED ──► PREPARING ──► READY ──► PICKED_UP     (F-04, then F-07)
PLACED ──reject──► REJECTED                                           (F-06)
PLACED ──cancel──► CANCELLED                                          (F-06)
```

The enum already holds every status so later sprints need no enum migration. Allowed moves live in one transition table read by `canTransition(from, to)` (DEC-05); Sprint 1 allows only `PLACED → ACCEPTED`. Payment status (F-08) will be a separate column (DEC-08).

3.4 Integrity rules kept by the database

Rule	How
One account per email	unique `users.email`
Order line copies survive menu changes (BR-05)	`name` and `unit_price` stored on `order_lines`; never updated
No duplicate item per order (BR-02)	unique (`order_id`, `menu_item_id`)
No partial order (BR-03, NFR-04)	order + lines inserted in one transaction
Accept at most once (BR-07)	`UPDATE orders SET status='ACCEPTED' … WHERE id=? AND status='PLACED'`; 0 rows → 409
Money exact (DEC-03)	integer satang everywhere; no floats

3.5 Migrations and seed

- Every schema change is a Prisma migration committed with the feature (`prisma migrate dev` locally; `prisma migrate deploy` in tests and production). Migrations only add things, so an older app version still runs on a newer schema.
- Seed (`server/prisma/seed.ts`, upsert, safe to repeat): users alice, bob (CUSTOMER), staff (OPERATOR) with password from `SEED_PASSWORD`; menu Latte 6500, Americano 5500, Thai Milk Tea 5000, Croissant 4500 (sold out), Ham & Cheese Sandwich 7500, Basil Chicken Rice 8000.
- Tests use a separate local `*_test` database, emptied before each test (`resetDb`).

4. API conventions

4.1 General

Topic	Convention
Base path	Everything under `/api`, same origin as the web app (no CORS)
Format	JSON request and response bodies, UTF-8; field names camelCase

Topic	Convention
Auth	`sid` httpOnly cookie (section 6); no tokens in headers or URLs
Money	Integer satang (`6500` = ฿65.00); the client only formats it
Time	ISO 8601 UTC strings (`2026-11-07T03:15:00.000Z`); the client shows local time
Ids	Positive integers in paths; anything else → 400 `VALIDATION_ERROR`
Lists	Plain JSON arrays, in a fixed documented order
Input checks	Zod schemas with `.strict()`: unknown fields are rejected (stops clients sending `price`, `total`, `role`, `customerId` — BR-04, NFR-02)
Success codes	200 read/update, 201 created, 204 no content (logout)
Unknown `/api/*` path	404 `NOT_FOUND`

4.2 Errors

Every error body is `{ "error": { "code", "message", "details" } }` (NFR-03). `message` is safe to show to the user; `details` is `null` or structured data the UI uses to mark fields or lines. No stack traces or SQL text are ever sent.

HTTP	`code`	When	Example `details`
400	`VALIDATION_ERROR`	Bad JSON, schema failure, bad id, business-rule failure on create (BR-01..BR-04)	Zod issues `[{ path, message }]`, or `{ items: [{ menuItemId, reason: "SOLD_OUT" | "NOT_FOUND" | "DUPLICATE" }] }`
401	`UNAUTHENTICATED`	No / unknown / expired session; wrong email or password	`null`
403	`FORBIDDEN`	Logged in with the wrong role	`null`
404	`NOT_FOUND`	Record missing **or** owned by another customer (DEC-07); unknown path	`null`
409	`INVALID_STATE`	Status change not allowed now (BR-06, BR-07)	`{ status }` current status
500	`INTERNAL_ERROR`	Anything unexpected (logged on the server only)	`null`

4.3 Shared JSON shapes

```
// User (login, me)
{ "id": 1, "name": "Alice", "role": "CUSTOMER" }

// Menu item
{ "id": 1, "name": "Latte", "price": 6500, "isAvailable": true }

// Order (POST /api/orders, GET /api/orders/:id, accept)
{
  "id": 7, "status": "PLACED", "total": 18000,
  "createdAt": "2026-11-07T03:15:00.000Z", "acceptedAt": null,
  "lines": [ { "menuItemId": 1, "name": "Latte", "unitPrice": 6500, "quantity": 2, "lineTotal": 13000 } ]
}

// Order summary (GET /api/orders list)
{ "id": 7, "status": "PLACED", "total": 18000, "itemCount": 3, "createdAt": "2026-11-07T03:15:00.000Z" }

// Queue order (GET /api/operator/orders): Order + customer
{ "...Order fields": "...", "customer": { "id": 1, "name": "Alice" } }
```

4.4 Sprint 1 endpoints

Details, all error cases and tests are in the feature contracts (`docs/sprints/sprint-1/F-0x-*.md`). Who may call each endpoint: section 5.2.

Method + path	Request	Success	Errors	Feature
`POST /api/auth/login`	`{ email, password }`	200 User + `Set-Cookie: sid`	400, 401	F-01
`POST /api/auth/logout`	—	204, cookie cleared	—	F-01
`GET /api/auth/me`	—	200 User	401	F-01
`GET /api/health`	—	200 `{ ok: true }` (public, for smoke tests)	—	#1
`GET /api/menu`	—	200 Menu item[] by `id`	401, 403	F-02
`POST /api/orders`	`{ items: [{ menuItemId, quantity }] }`	201 Order	400, 401, 403	F-03
`GET /api/orders`	—	200 Order summary[] newest first	401, 403	F-03
`GET /api/orders/:id`	—	200 Order	400, 401, 403, 404	F-03
`GET /api/operator/orders?status=PLACED`	—	200 Queue order[] oldest first	400, 401, 403	F-04
`POST /api/operator/orders/:id/accept`	—	200 Order (ACCEPTED, `acceptedAt` set)	400, 401, 403, 404, 409	F-04

Later sprints add endpoints with the same conventions (planned paths in section 5.2).

5. Access control

5.1 Principles

- The server decides who the user is (session cookie → `sessions` row → user) and what they may do. The role is never read from the request body, query or a client-side setting.
- Hiding a button in the UI is not authorization. Every protected endpoint has `requireAuth` or `requireRole`.
- No session or expired session → **401 UNAUTHENTICATED**. Logged in with the wrong role → **403 FORBIDDEN**. A customer asking for a record they do not own → **404 NOT_FOUND** (does not reveal it exists).

5.2 Role / permission matrix

✓ allowed · ✗ refused (code) · — not applicable

Operation	Endpoint	Anonymous	CUSTOMER	OPERATOR	Records the user may access	Sprint
Log in	`POST /api/auth/login`	✓	✓	✓	—	1
Log out	`POST /api/auth/logout`	✓ (no-op)	✓	✓	own session only	1
Who am I	`GET /api/auth/me`	✗ 401	✓	✓	own user (id, name, role)	1
View menu	`GET /api/menu`	✗ 401	✓	✗ 403	all menu items	1
Place order	`POST /api/orders`	✗ 401	✓	✗ 403	creates an order owned by the session user	1
List my orders	`GET /api/orders`	✗ 401	✓	✗ 403	only orders where `customerId` = session user	1
Order detail	`GET /api/orders/:id`	✗ 401	✓ own; other's → 404	✗ 403	only own orders	1
Order queue	`GET /api/operator/orders?status=PLACED`	✗ 401	✗ 403	✓	all orders (one shop)	1
Accept order	`POST /api/operator/orders/:id/accept`	✗ 401	✗ 403	✓	any PLACED order	1

Operation	Endpoint	Anonymous	CUSTOMER	OPERATOR	Records the user may access	Sprint
Manage menu / sold-out	`/api/operator/menu…`	✗ 401	✗ 403	✓	all menu items	2
Cancel order	`POST /api/orders/:id/cancel`	✗ 401	✓ own, PLACED only	✗ 403	own orders	2
Reject order	`POST /api/operator/orders/:id/reject`	✗ 401	✗ 403	✓	any PLACED order	2
Prepare / ready / picked up	`POST /api/operator/orders/:id/{prepare,ready,pickup}`	✗ 401	✗ 403	✓	any order in the right status	2
Pay (simulated)	`POST /api/orders/:id/pay`	✗ 401	✓ own	✗ 403	own orders	2
Sales summary	`GET /api/operator/sales`	✗ 401	✗ 403	✓	all orders (totals only)	3
Search orders	`GET /api/orders?…` / `GET /api/operator/orders?…`	✗ 401	✓ own	✓ all	as above	3

Sprint 2–3 paths are planned names; their feature contracts confirm them.

6. Authentication and sessions

Topic	Design
Accounts	Seeded only (no sign-up). Seed creates `alice@cafe.test`, `bob@cafe.test` (CUSTOMER) and `staff@cafe.test` (OPERATOR).
Passwords	Stored as bcrypt hashes (cost 10, `bcryptjs`). The plain password is never stored or logged. The seed password comes from env `SEED_PASSWORD`.
Login `POST /api/auth/login`	Body `{ email, password }` (strict Zod). Find user by email (case-insensitive), compare with bcrypt. Unknown email and wrong password give the **same** 401 "Incorrect email or password." On success: create a `sessions` row with id = 32 random bytes (`crypto.randomBytes`, base64url), `expiresAt` = now + 8 h; set cookie `sid`; return `{ id, name, role }`.
Cookie	`sid`, `HttpOnly`, `SameSite=Lax`, `Path=/`, `Max-Age` 8 h, `Secure` when `NODE_ENV=production`. The browser's JavaScript cannot read it.
Each request	`requireAuth` reads `sid`, loads the session with its user. Missing, unknown or expired (`expiresAt` ≤ now) → 401; an expired row is deleted. The user (id, role) is attached to the request for later checks.
Current user `GET /api/auth/me`	Returns `{ id, name, role }` of the session user, or 401.
Logout `POST /api/auth/logout`	Deletes the session row and clears the cookie. Returns 204 even if there was no session. Any later request with the old cookie → 401.
Fixed expiry	8 h from login, not extended by activity (simple to test, enough for a café shift).
Login rate limit	Sprint 3 (NFR-08).

7. Configuration and deployment

7.1 Environment variables (`server/.env`, from `server/.env.example`)

Variable	Used for	Local value	Production
`DATABASE_URL`	Prisma connection	`postgresql://cafe:cafe@localhost:5433/cafe` (Docker)	Cloud SQL connection, from Secret Manager
`TEST_DATABASE_URL`	Automated tests (must be local `*_test`)	`…/cafe_test`	not set

Variable	Used for	Local value	Production
`SEED_PASSWORD`	Password of seeded accounts	chosen by each developer	Secret Manager; never shown on screen
`PORT`	HTTP port	3000	set by Cloud Run
`NODE_ENV`	`production` turns on the `Secure` cookie flag and serving the built client	unset	`production`

7.2 Secrets

- Real values live only in `server/.env` (git-ignored) locally and in **Secret Manager** in GCP.
- `.env.example` holds placeholders only. No secret is in Git, the client bundle, logs or the PDF.
- The client has no secrets: it only calls same-origin `/api`.
- Tests refuse to run against any database that is not local and named `*_test`.

7.3 Database setup

- Local: `docker compose up -d` (Postgres 16, port 5433, named volume `pgdata` so data survives restarts).
- Schema changes only through Prisma migrations in `server/prisma/migrations/`, committed to Git. `prisma migrate dev` locally, `prisma migrate deploy` in tests and production. Migrations are additive so a rollback of the app does not need a schema rollback.
- Seed (`npm run db:seed`) is repeatable (upsert): same 3 users and 6 menu items every time.

7.4 Planned GCP deployment (Lab 5, Sprint 2)

```
GitHub (PR → required CI check; merge to main → deploy workflow)
    │ build image tagged with commit SHA
    ▼
Artifact Registry ──► Cloud Run service (one container: Express + built React, port from $PORT)
                        │            ▲
                        │            └─ Secret Manager: DATABASE_URL, SEED_PASSWORD
                        ▼
                    Cloud SQL for PostgreSQL 16 (durable storage, daily backups)
Cloud Run job "migrate": prisma migrate deploy + seed, run before each release
```

Part	Plan
Image	Multi-stage Dockerfile (from the Lab 5 kit): build client, build server, final image with `node` non-root user.
Runtime	Cloud Run, region `asia-southeast1`. Stateless: sessions are in the database, so any instance can serve any request.
Storage	Cloud SQL PostgreSQL 16. No files on the container disk.
Releases	GitHub Actions: CI on every PR (tests + Docker build); on merge to `main` build, push, run migrate job, deploy by image digest, smoke test `/api/health`. Rollback = redeploy an earlier image.
Cost / safety	Budget alert first; teardown script after the course.

Change log

Date	Version	Change	Why
6 Nov 2026	1	Sections 1, 2, 5, 6, 7 (A); 3, 4 placeholders for B	Deliverable 1 part C
6 Nov 2026	1.1	Sections 3 data model and 4 API conventions (B)	Deliverable 1 part C

Part D — Three-sprint plan

Three-sprint plan — Campus Café Pre-order

Deliverable 1, part D. Issue #9. Source of decisions: `docs/PLAN.md`. Version 1 — 6 Nov 2026. Changes after this are recorded in the change log at the bottom, not by rewriting this text.

1. Sprint goals and dates

Sprint	Due (23:59)	Goal	Features	Release tag
1	Sun 8 Nov 2026	A customer can log in, order from the menu and see the order; staff can accept it. Login and roles are enforced by the server.	F-01, F-02, F-03, F-04	`v0.1.0-sprint1`
2	Sun 29 Nov 2026	The café can run a full day: edit the menu, cancel or reject orders, move orders through preparation to pick-up, and record (simulated) payment.	F-05, F-06, F-07, F-08	`v0.2.0-sprint2`
3	Sun 13 Dec 2026	Finish and harden: pickup time slots, sales summary, order search, rate limit, error states and accessibility.	F-09, F-10, F-11, F-12	`v1.0.0`

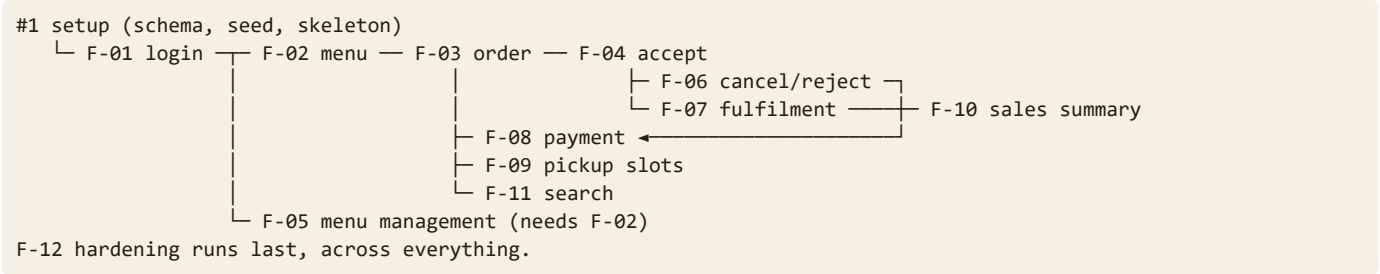
Course labs use the same app: Lab 5 (Docker, CI/CD, Google Cloud deploy) due 15 Nov, Lab 6 (cross-team user acceptance testing) due 22 Nov, Lab 7 (maintenance change) due 29 Nov. Lab work is planned inside Sprint 2 time but kept in its own issues.

2. Features by sprint

ID	Feature	Sprint	Owner	Depends on
F-01	Login, logout, "who am I" (CUSTOMER, OPERATOR)	1	A (vieng)	#1 setup
F-02	Browse menu (sold-out items greyed out)	1	B (ninja)	F-01
F-03	Place order, my orders, order detail	1	B (ninja)	F-01, F-02
F-04	Operator queue + accept order	1	A (vieng)	F-01, F-03
F-05	Menu management + sold-out toggle	2	to agree	F-02
F-06	Customer cancel / operator reject with reason	2	to agree	F-04
F-07	Fulfilment: PREPARING → READY → PICKED_UP	2	to agree	F-04
F-08	Simulated payment (UNPAID → PAID → REFUNDED)	2	to agree	F-03, F-06
F-09	Pickup time slots + order cut-off	3	to agree	F-03
F-10	Sales summary	3	to agree	F-07, F-08
F-11	Order search / filter	3	to agree	F-03, F-07
F-12	Hardening: login rate limit, error states, accessibility	3	both	all

Sprint 2 and 3 owners are agreed at the start of each sprint; each person keeps a mix of backend, frontend, tests and docs.

3. Dependencies



Notes:

- F-03 and F-04 share the order status table. Sprint 1 builds only `PLACED → ACCEPTED` with a transition table and `canTransition()`; F-06 and F-07 add rows to the same table instead of new code paths.
- F-02 and F-03 can be built in parallel with F-01 by using a test login helper; they are merged after F-01.

4. Sprint 1 — why it is feasible

Scope is the smallest slice the course asks for (spec section 6, theme 4): login with two roles, a seeded menu, one order transaction, one state change, and tested business rules BR-01..BR-07.

Task	Issue	Owner	When
Schema, seed, skeleton, test setup	#1	both	Fri night
Doc A SRS	#6	A	Fri night draft, Sat final
Doc B features + traceability	#7	B	Fri night draft, Sun final (needs test IDs)
Doc C system SDS	#8	A: auth, roles, config; B: data model, API	Fri night draft, Sat final
Doc D this plan	#9	B	Fri night
Contracts (SDS, AC, test spec, Definition of Done) F-01..F-04	#2–#5	owner of each feature	Sat morning, committed before code
Tests first, then code, F-01..F-04	#2–#5	owner of each feature	Sat afternoon / night
End-to-end tests (Playwright)	#10	A	Sun morning
Release PR `sprint-1` → `main`, tag	#11	A	Sun afternoon
PDF, screenshots, test output	#11	B	Sun evening, submit by ~21:00

Every pull request is reviewed and merged by the other person, so both must be online Saturday and Sunday (agreed in chat 6 Nov).

5. Remaining work after Sprint 1

- Sprint 2: F-05..F-08, plus Lab 5 (container, CI with a required check, deploy from `main`) and Lab 6 fixes.
- Sprint 3: F-09..F-12, Lab 7 change, regression tests, final report and demo video.
- Contracts for Sprint 2 features are written at the start of Sprint 2; Sprint 3 contracts are due with Deliverable 2 (29 Nov).

6. Risks

Risk	Effect	What we do
Only one weekend for Sprint 1	Late or untested features	Keep Sprint 1 to F-01..F-04; no extra features
Reviews need the partner online	Pull requests wait	Agreed weekend times; small pull requests
Two staff accept at once (BR-07)	Order accepted twice	Conditional update `where status = PLACED` + concurrency test
Exam period 1–11 Dec	Sprint 3 squeezed	Product mostly done by 29 Nov; after that only fixes
Cloud costs / access (Lab 5)	Deploy blocked	Budget alert; follow the Lab 5 starter files

Change log

Date	Change	Why
6 Nov 2026	First version	Deliverable 1 part D

Part E — Sprint 1 engineering contracts

F-01 Login, logout, current user — engineering contract

Sprint 1. Issue #2. Owner: A (vieng). Reviewer: B (ninja). Branch `feature/F-01-auth`. Definition of Done: `definition-of-done.md`. Status: **Specified** (committed before any F-01 code).

1. Feature and requirements

Linked	Text (short)
FR-01	Log in with email + password, land on the role's home page
FR-02	Log out; old session no longer works
FR-03	Current user (name, role) for the header and redirects
FR-04	Server checks session and role on every protected endpoint
NFR-01	bcrypt hashes; random session id in httpOnly SameSite=Lax cookie (Secure in production); 8 h expiry; logout deletes it
NFR-02	Role never taken from the request
NFR-03	Shared error format

Depends on: Issue #1 (schema `users`, `sessions`; seed accounts; `AppError`). Used by: F-02, F-03, F-04 (`requireAuth`, `requireRole`).

2. Behaviour

Workflow

1. Visitor opens the app → client calls `GET /api/auth/me` → 401 → shows `/login`.
2. Visitor submits email + password → `POST /api/auth/login`.
3. Server lower-cases the email, finds the user, compares the password with the bcrypt hash.
4. Match → create `sessions` row (id = 32 random bytes, base64url; `expiresAt` = now + 8 h), set cookie `sid`, return the user. No match → 401, nothing created.
5. Client stores the returned user in memory and redirects: CUSTOMER → `/menu`, OPERATOR → `/operator/orders`.
6. Every protected request: `requireAuth` reads `sid` → loads session + user → not found or expired → 401 (expired row deleted). `requireRole(role)` → other role → 403.
7. Log out → `POST /api/auth/logout` → row deleted, cookie cleared → client goes to `/login`.

Rules, validation, boundaries

- Body must be exactly `{ email, password }`: email a valid address, password a non-empty string (max 200 chars). Any other field (for example `role`) → 400. Nothing is created.
- Unknown email and wrong password give the **same** status and body.
- Session expiry is fixed at 8 h from login (not extended). `expiresAt <= now` counts as expired.
- Logout without a session is not an error (204).
- Response bodies never contain `passwordHash` or the session id.

Permissions

Endpoint	Anonymous	CUSTOMER	OPERATOR
`POST /api/auth/login`	✓	✓	✓
`POST /api/auth/logout`	✓ (204, no-op)	✓	✓
`GET /api/auth/me`	401	✓ own	✓ own

3. UI

Screens per `../../ui/ui-spec.md` (S1 Login, shared layout, route guard).

Part	Behaviour
S1 Login `/login`	Email, Password, **Log in**. Client checks: "Email is required." / "Enter a valid email." / "Password is required." Loading: "Logging in…" (disabled). 401: error box "Incorrect email or password.", password cleared, email kept. Network/500: "Something went wrong. Please try again." Already logged in → redirect to own home. Arrived after a 401 elsewhere: info box "Your session has ended. Please log in again."
Shared layout	Phone: top bar "Hey, `<name>`", role chip, Log out button; bottom tab bar by role. Desktop: dark sidebar with links by role, name and role, Log out. Log out shows "Logging out…", then always goes to `/login`.
Route guard	On start call `/api/auth/me`. No session → `/login`. Customer on an operator route (or the reverse) → own home with "You don't have access to that page." `/` → own home. Unknown path → "Page not found" + Go home.

4. API

All errors use `{ error: { code, message, details } }`.

Method + path	Request	Success	Errors
`POST /api/auth/login`	`{ "email": "alice@cafe.test", "password": "…" }`	200 `{ "id": 1, "name": "Alice", "role": "CUSTOMER" }` + `Set-Cookie: sid=…; Path=/; Max-Age=28800; HttpOnly; SameSite=Lax` (+ `Secure` in production)	400 VALIDATION_ERROR (bad/extra fields) · 401 UNAUTHENTICATED "Incorrect email or password."
`GET /api/auth/me`	cookie `sid`	200 `{ "id", "name", "role" }`	401 UNAUTHENTICATED "Not logged in."
`POST /api/auth/logout`	cookie `sid` (optional)	204, cookie cleared	—

Middleware for other features: `requireAuth` (401 "Not logged in.") and `requireRole("CUSTOMER" | "OPERATOR")` (401, or 403 FORBIDDEN "You don't have access to this.").

5. Data

No schema change. Uses `users` (email unique, `password_hash`, `role`) and `sessions` (`id` PK, `user_id` FK cascade, `expires_at`) from Issue #1 migration `init`. Seed accounts from Issue #1.

6. Acceptance criteria

ID	Given / when	Then
AC-F01-01	Seeded customer; POST login with correct email + password	200 `{ id, name, role: "CUSTOMER" }`; cookie `sid` is HttpOnly, SameSite=Lax, Max-Age 28800; one `sessions` row for that user with `expiresAt` 8 h ± 1 min from now
AC-F01-02	Correct email, wrong password	401 UNAUTHENTICATED "Incorrect email or password."; no `Set-Cookie`; no session row
AC-F01-03	Email that has no account	Same status and body as AC-F01-02; no session row
AC-F01-04	Body missing password, invalid email, or with an extra field `role: "OPERATOR"`	400 VALIDATION_ERROR; no session row

ID	Given / when	Then
AC-F01-05	Email typed in capitals `ALICE@CAFE.TEST`	Login succeeds as Alice
AC-F01-06	Valid session; GET me	200 `{ id, name, role }` only — no `passwordHash`, no session id
AC-F01-07	GET me with no cookie, or with a made-up `sid`	401 UNAUTHENTICATED
AC-F01-08	Session whose `expiresAt` has passed; GET me	401 and that session row is deleted
AC-F01-09	Logged in; POST logout, then GET me with the old cookie	Logout 204 and cookie cleared; row deleted; then 401
AC-F01-10	A route protected by `requireRole("OPERATOR")`	Customer session → 403 FORBIDDEN; no session → 401; operator → 200
AC-F01-11	`NODE_ENV=production`	Session cookie also has `Secure`
AC-F01-12	UI: customer logs in / operator logs in / user logs out	Customer lands on `/menu`, operator on `/operator/orders`; layout shows the user's name and role; after Log out the app shows `/login` and `/menu` redirects to `/login`
AC-F01-13	UI: wrong password	Error "Incorrect email or password." shown; password field empty; email field keeps its value
AC-F01-14	UI: not logged in opens `/menu`; customer opens `/operator/orders`	First → `/login`; second → `/menu` with "You don't have access to that page."

7. Test specification

Level: **API** = Vitest + Supertest against the local `cafe_test` Postgres; **unit** = Vitest, no database; **E2E** = Playwright in a browser with the seeded database. The test ID is at the start of each test name.

Test ID	AC	Level	Precondition / input	Action	Expected
TEST-F01-01	AC-F01-01	API	DB reset + seed (test password)	POST `/api/auth/login` alice + correct password	200 body; cookie flags; 1 session row, expiry 8 h ± 1 min
TEST-F01-02	AC-F01-02	API	Seeded	POST login alice + wrong password	401 body as specified; no `set-cookie`; 0 session rows
TEST-F01-03	AC-F01-03	API	Seeded	POST login `nobody@cafe.test`	Body equals TEST-F01-02 body; 0 session rows
TEST-F01-04	AC-F01-04	API	Seeded	POST login with (a) no password (b) `email: "not-an-email"` (c) extra `role`	400 VALIDATION_ERROR each; 0 session rows
TEST-F01-05	AC-F01-05	API	Seeded	POST login `ALICE@CAFE.TEST`	200, name Alice

Test ID	AC	Level	Precondition / input	Action	Expected
TEST-F01-06	AC-F01-06	API	Logged in as staff	GET `/api/auth/me` with cookie	200 `{ id, name: "Staff", role: "OPERATOR" }`, keys exactly id/name/role
TEST-F01-07	AC-F01-07	API	—	GET me (a) no cookie (b) `sid=made-up`	401 UNAUTHENTICATED both
TEST-F01-08	AC-F01-08	API	Session row with `expiresAt` 1 s in the past	GET me with that `sid`	401; row no longer exists
TEST-F01-09	AC-F01-09	API	Logged in as bob	POST logout; GET me with old cookie	204 + cookie cleared; 0 rows for bob; then 401
TEST-F01-10	AC-F01-10	API	Test-only route with `requireRole("OPERATOR")`	Call as customer, anonymous, operator	403, 401, 200
TEST-F01-11	AC-F01-11	unit	—	`sessionCookieOptions(true)` and `(false)`	`secure: true` / `secure: false`; both `httpOnly`, `sameSite: "lax"`, `maxAge` 8 h
TEST-F01-12	AC-F01-12	E2E	Seeded DB, app running	Log in as alice → check URL + name; log out → open `/menu`; log in as staff → check URL	`/menu` + "Alice"; `/login`; `/operator/orders` + "Staff"
TEST-F01-13	AC-F01-13	E2E	Seeded	Log in alice + wrong password	Error text visible; password input empty; email input = alice@cafe.test
TEST-F01-14	AC-F01-14	E2E	Seeded	Open `/menu` logged out; log in alice, open `/operator/orders`	`/login`; `/menu` + "You don't have access to that page."

TDD: TEST-F01-01..11 are committed failing first (`[red]`, output saved to `docs/evidence/tdd/F-01-red.txt`), then the code that makes them pass (`[green]`, `F-01-green.txt`). E2E tests run in Issue #10 once the screens exist.

8. Definition of Done

See `definition-of-done.md`. F-01 specific: the middleware is used by F-02..F-04 endpoints.

9. Version history

Date	Version	Change	Why
6 Nov 2026	1	First contract	Before implementation (Issue #2)

F-02 Browse menu — feature contract

Sprint 1 · Issue #3 · Owner: ninja (B) · Reviewer: vieng (A) Status: Specified (committed before any F-02 code)

1. Feature and linked requirements

A logged-in customer sees the café menu: every item with its name and price. Sold-out items stay visible but are greyed out and cannot be added to the cart.

Type	IDs
Functional	FR-05 view menu with price and availability; FR-04 role check on the endpoint
Business rules	BR-03 (sold-out items cannot be ordered — shown here, enforced again by F-03 on submit)
Non-functional	NFR-02 prices only from the server, NFR-03 error format, NFR-05 tests first, NFR-06 loading/empty/error states, phone + desktop
Depends on	#1 setup (`menu_items` table, seed), F-01 (session and `requireRole`)

2. Behaviour, workflow and permissions

1. Customer opens `/menu` (their home page after login).
2. The app calls `GET /api/menu`.
3. The server checks the session (F-01) and the role, reads all rows of `menu_items` ordered by `id`, and returns them.
4. The app shows each item; items with `isAvailable = false` are greyed with a **Sold out** badge and a disabled **+** button.
5. Adding to the cart (client-side cart) is part of F-03.

Who	`GET /api/menu`
Not logged in / session expired	401 `UNAUTHENTICATED`
CUSTOMER	200
OPERATOR	403 `FORBIDDEN` (operators manage the menu with F-05 in Sprint 2)

Rules:

- Prices are integers in satang, returned exactly as stored. The client only formats them (`6500` → `฿65.00`).
- Sold-out items are **included** in the response (not filtered out), so the customer sees what exists.
- The menu is read-only in Sprint 1.

3. UI (screen S2, menu list part)

Source: `docs/ui/ui-spec.md` S2 and `docs/ui/design/Menu.html`, `DesktopMenu.html`.

- Banner: "Order ahead. Skip the line." / "Coffee and snacks from ฿35. Pickup ASAP at the counter."
- Phone: one row per item. Desktop: 3-column cards.
- Each item: name, price `฿xx.xx`, round **+** button with accessible label "Add `<item name>`".
- Sold out: greyed, **Sold out** badge, + disabled.
- In cart (from F-03): "In cart · N" badge.

State	What the user sees
Loading	Grey placeholder rows; no + buttons
Empty (0 items)	"Nothing on the menu yet. Please check back later."
Error (network / 500)	"Couldn't load the menu. Check your connection." + **Reload menu**
401	Shared rule: clear user, go to `/login`, "Your session has ended. Please log in again."
403	Shared rule: go to own home, "You don't have access to that page."

State	What the user sees
Success	Menu list as above

4. API

`GET /api/menu`

Request: no body, no query. Session cookie required.

Response 200:

```
[
  { "id": 1, "name": "Latte", "price": 6500, "isAvailable": true },
  { "id": 4, "name": "Croissant", "price": 4500, "isAvailable": false }
]
```

- A plain array (same list style as F-03/F-04), ordered by `id` ascending; `[]` when there are no rows.
- Only these four fields are returned.

Errors (shared shape `{ error: { code, message, details } }`):

HTTP	code	When
401	`UNAUTHENTICATED`	No session cookie, unknown or expired session
403	`FORBIDDEN`	Logged in as OPERATOR
500	`INTERNAL_ERROR`	Unexpected error (no stack trace)

5. Data

No schema change. Reads `menu_items` (`id`, `name`, `price` Int satang, `is_available` Boolean) created in #1. Seed has 6 items, one sold out (Croissant).

6. Acceptance criteria

ID	Given / When / Then
AC-F02-01	Given a logged-in customer, when they request the menu, then they get 200 and every menu item with `id`, `name`, `price` (integer satang) and `isAvailable`, ordered by `id`.
AC-F02-02	Given an item marked sold out, when the menu is requested, then that item is still in the list with `isAvailable: false`.
AC-F02-03	Given no session (or an expired one), when the menu is requested, then the server answers 401 `UNAUTHENTICATED` in the shared error shape.
AC-F02-04	Given a logged-in operator, when they request the menu, then the server answers 403 `FORBIDDEN`.
AC-F02-05	Given an empty `menu_items` table, when a customer requests the menu, then they get 200 with `[]`.
AC-F02-06	Given the customer opens `/menu`, then each item shows its name and price as `฿xx.xx`, and a sold-out item is greyed with a **Sold out** badge and a disabled + button.
AC-F02-07	Given the menu request fails (network or 500), then the page shows "Couldn't load the menu. Check your connection." and **Reload menu** loads it again.

7. Test specification

Test ID	AC	Level	Precondition / input	Action	Expected
TEST-F02-01	AC-F02-01	API	Seeded DB; logged in as alice (CUSTOMER)	`GET /api/menu`	200; 6 items; ids ascending; each has exactly `id, name, price, isAvailable`; every `price` is a positive integer; Latte price = 6500
TEST-F02-02	AC-F02-02	API	Seeded DB (Croissant sold out); logged in as alice	`GET /api/menu`	Croissant present with `isAvailable: false`; other 5 `true`
TEST-F02-03	AC-F02-03	API	No cookie	`GET /api/menu`	401, `error.code = UNAUTHENTICATED`, `message` is a string
TEST-F02-04	AC-F02-03	API	Session row with `expiresAt` in the past	`GET /api/menu` with that cookie	401 `UNAUTHENTICATED`
TEST-F02-05	AC-F02-04	API	Logged in as staff (OPERATOR)	`GET /api/menu`	403, `error.code = FORBIDDEN`
TEST-F02-06	AC-F02-05	API	Users seeded, `menu_items` empty; logged in as alice	`GET /api/menu`	200, `[]`
TEST-F02-07	AC-F02-06	unit	`formatBaht(6500)`, `formatBaht(15500)`, `formatBaht(0)`, `formatBaht(5)`	call	`฿65.00`, `฿155.00`, `฿0.00`, `฿0.05`
TEST-F02-08	AC-F02-06	E2E	Seeded DB; log in as alice in the browser	open `/menu`	Latte shows `฿65.00`; Croissant shows **Sold out** and its "Add Croissant" button is disabled
TEST-F02-09	AC-F02-07	E2E	Logged in as alice; `/api/menu` answered with 500 (route mocked)	open `/menu`, then click **Reload menu** after the mock is removed	Error text shown; after reload the menu list appears

Tests are written first and must fail before the code exists (`[red]` commit, output saved in `docs/evidence/tdd/`), then pass (`[green]` commit). Test names start with the test ID, e.g. `it("TEST-F02-01 ...")`.

8. Definition of Done

Shared checklist: `definition-of-done.md`. F-02 extra evidence: red and green test output for TEST-F02-01..06, E2E run for TEST-F02-08..09, screenshot of `/menu` on phone and desktop with the sold-out item.

9. Version history

Version	Date	Change	By
1	6 Nov 2026	First version, before implementation	ninja
1.1	6 Nov 2026	Response is a plain array (list style agreed with F-03/F-04 in chat). Still before implementation	ninja

F-03 Place order, my orders, order detail — feature contract

Sprint 1 · Issue #4 · Owner: ninja (B) · Reviewer: vieng (A) Status: Specified (committed before any F-03 code)

1. Feature and linked requirements

A customer builds a cart from the menu and places an order. The server checks every rule, calculates the total from database prices, and saves the order with its lines in one transaction. The customer can list their own orders and open one to see its lines, total and status.

Type	IDs
Functional	FR-06 place an order, FR-07 list my orders, FR-08 view my order detail; FR-04 role check
Business rules	BR-01, BR-02, BR-03, BR-04, BR-05
Non-functional	NFR-02 prices only from the server, NFR-03 error format, NFR-04 one transaction per order, NFR-05 tests first, NFR-06 states, phone + desktop
Depends on	#1 setup (`orders`, `order_lines`), F-01 (session, `requireRole`), F-02 (`GET /api/menu`, `formatBaht`)
Used by	F-04 (operator sees and accepts PLACED orders)

2. Behaviour, workflow and permissions

Place order

1. On `/menu` the customer adds items to a cart kept in the browser (client state only; not saved).
2. **Place order** sends `POST /api/orders` with only `menuItemId` and `quantity` per line.
3. The server, in this order:
    1. checks the session and role (401 / 403);
    2. validates the body shape with a strict schema (BR-01, BR-02, BR-04) → 400 on any problem;
    3. in **one database transaction**: loads the menu items, checks every item exists and is available (BR-03), calculates `unitPrice × quantity` per line and the total from database prices (BR-04), creates the order with status `PLACED` and its lines with a copy of `name` and `unitPrice` (BR-05);
    4. returns 201 with the created order.
4. If any check fails, nothing is saved (no order, no lines).
5. The app clears the cart and goes to `/orders/:id` with "Order #N placed!".

My orders / order detail

- `GET /api/orders` returns only the logged-in customer's orders, newest first.
- `GET /api/orders/:id` returns the order only if it belongs to the logged-in customer. A missing id and another customer's order both give **404** with the same message, so a customer cannot tell whether someone else's order exists.

Who	`POST /api/orders`	`GET /api/orders`	`GET /api/orders/:id`
Not logged in / expired	401	401	401
CUSTOMER	201 / 400	200 (own only)	200 own, 404 otherwise
OPERATOR	403	403	403 (operators use F-04 endpoints)

Business rules (server is the source of truth; the client checks are only for convenience)

Rule	Server check	Error
BR-01	Each `quantity` is a whole number 1–10	400 `VALIDATION_ERROR`
BR-02	`items` has 1–20 lines; no `menuItemId` appears twice	400 `VALIDATION_ERROR`

Rule	Server check	Error
BR-03	Every `menuItemId` exists and `isAvailable = true`; else whole order rejected, nothing saved	400 `VALIDATION_ERROR`
BR-04	Total and line prices come from `menu_items`; any extra field (e.g. `price`, `unitPrice`, `total`) anywhere in the body is rejected	400 `VALIDATION_ERROR`
BR-05	`order_lines.name` and `order_lines.unit_price` are copied at order time and never re-read from the menu	—

3. UI

Source: `docs/ui/ui-spec.md` S2 (cart), S3, S4 and `docs/ui/design/Menu.html`, `Orders.html`, `DesktopMenu.html`, `DesktopOrders.html`.

S2 cart (on `/menu`)

- Line: name, stepper (− qty +), line subtotal, ✕ remove. "Pickup: ASAP". "Estimated total". **Place order** →.
- Phone: cart is its own view, opened from the "N items · ฿x · View cart →" bar or the Cart tab. Desktop: panel on the right.
- **+** on an item already in the cart adds 1 to that line (no duplicate line); the item shows "In cart · N".
- Stepper − disabled at 1 ("Min 1 — use ✕ to remove."), + disabled at 10 ("Max 10 per item."); typed values are clamped to 1–10.
- At 20 lines, + on a new item is disabled with "Cart is full (20 different items max)."

State	What the user sees
Empty cart	"Your cart is empty. Add something from the menu." Place order disabled
Submitting	"Placing order…"; steppers and button disabled
400	`error.message` in an error box; menu reloaded; bad lines from `error.details.items` marked; cart kept
Network / 500	"Something went wrong. Please try again." + **Try again**; cart kept
Success	Cart cleared → `/orders/:id` with "Order #N placed!"

S3 My orders (`/orders`)

- Newest first: `#id`, placed time, item count, status pill (`PLACED` orange, `ACCEPTED` green), total `฿xx.xx`. Whole card/row opens `/orders/:id`. **Refresh list**.
- Loading: placeholder cards. Empty: "No orders yet. Your orders will show up here." + **Browse the menu**. Error: "Couldn't load your orders." + **Reload orders**.

S4 Order detail (`/orders/:id`)

- "← My orders", `#id`, status badge, placed time, "Pickup: ASAP", lines (name, unit price, × qty, line total), **Total** from the server, status hint ("Waiting for the café to accept your order." / "Accepted — being prepared soon."), **Refresh status**. Desktop timeline: Placed (time) → Accepted / "Waiting for the café".
- Loading: placeholder rows. 404: "Order not found." + Back to my orders. Error: "Couldn't load this order." + **Reload order**.

Times: today → `10:15`, yesterday → `Yesterday 12:02`, older → `3 Nov 09:40` (local time).

4. API

`POST /api/orders`

Request (strict: no other keys allowed at any level):

```
{ "items": [ { "menuItemId": 1, "quantity": 2 }, { "menuItemId": 3, "quantity": 1 } ] }
```

Response **201** — the created order (same shape as `GET /api/orders/:id`):

```
{
  "id": 7,
  "status": "PLACED",
  "total": 18000,
  "createdAt": "2026-11-07T03:15:00.000Z",
  "lines": [
    { "menuItemId": 1, "name": "Latte", "unitPrice": 6500, "quantity": 2, "lineTotal": 13000 },
    { "menuItemId": 3, "name": "Thai Milk Tea", "unitPrice": 5000, "quantity": 1, "lineTotal": 5000 }
  ]
}
```

Errors:

HTTP	code	When	`message` / `details`
400	`VALIDATION_ERROR`	Body shape wrong: missing/empty `items`, more than 20 lines, `quantity` not an integer 1–10, `menuItemId` not a positive integer, unknown key such as `price` or `total`	`message`: "Invalid request." (from the shared Zod handler); `details`: the list of problems with their path
400	`VALIDATION_ERROR`	Same `menuItemId` twice (BR-02)	"Order not placed. Each item can appear only once."; `details: { items: [{ menuItemId, reason: "DUPLICATE" }] }`
400	`VALIDATION_ERROR`	Item does not exist or is sold out (BR-03)	"Order not placed. 'Croissant' is no longer available." (names of all bad items, or "an item" when unknown); `details: { items: [{ menuItemId, reason: "NOT_FOUND" | "SOLD_OUT" }] }`
401	`UNAUTHENTICATED`	No or expired session	
403	`FORBIDDEN`	OPERATOR	

`GET /api/orders`

Response 200: a plain array, newest first (`createdAt` desc, then `id` desc):

```
[ { "id": 7, "status": "PLACED", "total": 18000, "itemCount": 3, "createdAt": "2026-11-07T03:15:00.000Z" } ]
```

`itemCount` = sum of quantities. Errors: 401, 403.

`GET /api/orders/:id`

Response 200: same shape as the `POST` response (the shared order shape, also used by F-04). Errors: 401, 403, **400** `VALIDATION_ERROR` when the id is not a positive whole number, **404** `NOT_FOUND` **"Order not found."** when it does not exist or belongs to another customer.

5. Data

No schema change; uses tables from #1.

- `orders`: `customer_id` = session user, `status` = `PLACED`, `total` = server sum (Int satang), `created_at`.
- `order_lines`: `order_id`, `menu_item_id`, `name` and `unit_price` copied from `menu_items` at order time, `quantity`. Unique (`order_id`, `menu_item_id`) backs BR-02 in the database.
- The order and all its lines are written with one `prisma.$transaction`, so a failure leaves no partial order.

6. Acceptance criteria

ID	Given / When / Then
AC-F03-01	Given a logged-in customer and available items, when they place an order with 1–20 lines of quantity 1–10, then the server answers 201 with status `PLACED`, the lines, and a total equal to the sum of database price × quantity; the order and lines are saved.
AC-F03-02	(BR-01) When any line has quantity 0, 11, a decimal, a negative number or a non-number, then 400 `VALIDATION_ERROR` and nothing is saved. Quantities 1 and 10 are accepted.
AC-F03-03	(BR-02) When `items` is empty or has 21 lines, or the same item appears twice, then 400 `VALIDATION_ERROR` and nothing is saved. 20 different lines are accepted.
AC-F03-04	(BR-03) When any item is sold out or does not exist, then 400 `VALIDATION_ERROR` naming the item(s) in `details.items`, and no order or line is saved, even if other lines were valid.
AC-F03-05	(BR-04) When the request contains `price`, `unitPrice` or `total` (top level or in a line), then 400 `VALIDATION_ERROR` and nothing is saved; a valid order's total never depends on client values.
AC-F03-06	(BR-05) Given a placed order, when the menu price or name changes later, then the order detail still shows the name, unit price and total from order time.
AC-F03-07	When a customer lists orders, then they get only their own orders, newest first, with id, status, total, item count and time.
AC-F03-08	When a customer opens their own order, then they get 200 with lines and total; when they open another customer's order or a missing id, then 404 `NOT_FOUND` with the same message; a malformed id (e.g. `abc`) gives 400 `VALIDATION_ERROR`.
AC-F03-09	When not logged in, any F-03 endpoint answers 401; when logged in as operator, 403.
AC-F03-10	In the browser, a customer adds items (adding the same item twice gives quantity 2 on one line), places the order, lands on `/orders/:id` with "Order #N placed!", and sees the server total.
AC-F03-11	In the browser, when the server rejects the order with 400, the message is shown, the cart is kept, and the bad line is marked.
AC-F03-12	In the cart, the stepper cannot go below 1 or above 10, and a 21st different item cannot be added.

7. Test specification

Seed: alice, bob (CUSTOMER), staff (OPERATOR); Latte 6500 (id 1), Americano 5500, Thai Milk Tea 5000, Croissant 4500 sold out, Sandwich 7500, Basil Chicken Rice 8000. Tests that need 20+ items create extra menu rows.

Test ID	AC	Level	Precondition / input	Action	Expected
TEST-F03-01	AC-F03-01	API	alice; Latte ×2, Thai Milk Tea ×1	POST	201; status PLACED; total 18000; 2 lines with name + unitPrice; 1 order and 2 lines in DB for alice
TEST-F03-02	AC-F03-02	API	alice; quantity 1 and 10	POST	201

Test ID	AC	Level	Precondition / input	Action	Expected
TEST-F03-03	AC-F03-02	API	alice; quantity 0, 11, 1.5, -1, "2" (one request each)	POST	each 400 `VALIDATION_ERROR`; order count unchanged
TEST-F03-04	AC-F03-03	API	alice; `items: []`; missing `items`	POST	400; nothing saved
TEST-F03-05	AC-F03-03	API	alice; 20 different available items ×1	POST	201 with 20 lines
TEST-F03-06	AC-F03-03	API	alice; 21 different items	POST	400; nothing saved
TEST-F03-07	AC-F03-03	API	alice; Latte ×1 twice	POST	400; `details.items[0]` = `{ menuItemId: 1, reason: "DUPLICATE" }`; nothing saved
TEST-F03-08	AC-F03-04	API	alice; Latte ×1 + Croissant ×1	POST	400; message names Croissant; `reason: "SOLD_OUT"`; no order and no lines saved
TEST-F03-09	AC-F03-04	API	alice; menuItemId 9999	POST	400; `reason: "NOT_FOUND"`; nothing saved
TEST-F03-10	AC-F03-05	API	alice; line with `price: 1`; body with `total: 1`; line with `unitPrice: 1`	POST	each 400; nothing saved
TEST-F03-11	AC-F03-06	API	alice places Latte ×1; then Latte price set to 9900 and name to "Latte XL" in DB	GET `/api/orders/:id`	line name "Latte", unitPrice 6500, total 6500
TEST-F03-12	AC-F03-07	API	alice has 2 orders, bob has 1	alice GET `/api/orders`	2 orders, newest first, both alice's; `itemCount` correct
TEST-F03-13	AC-F03-08	API	alice has order A	alice GET `/api/orders/A`	200 with lines and total
TEST-F03-14	AC-F03-08	API	bob has order B	alice GET `/api/orders/B`; alice GET `/api/orders/99999`	both 404 `NOT_FOUND`, same message
TEST-F03-21	AC-F03-08	API	alice	GET `/api/orders/abc`, `/api/orders/0`, `/api/orders/-1`	each 400 `VALIDATION_ERROR`
TEST-F03-15	AC-F03-09	API	no cookie	POST, GET list, GET detail	all 401
TEST-F03-16	AC-F03-09	API	staff (OPERATOR)	POST, GET list, GET detail	all 403; nothing saved
TEST-F03-17	AC-F03-01	unit	`cartReducer`: add Latte, add Latte, add Americano	apply	2 lines; Latte qty 2
TEST-F03-18	AC-F03-12	unit	`cartReducer`: set qty 0, 11, 2.7; add 21st item	apply	qty clamped to 1, 10, 2; 21st item refused

Test ID	AC	Level	Precondition / input	Action	Expected
TEST-F03-19	AC-F03-10	E2E	alice logged in	add Latte twice + Thai Milk Tea, Place order	URL `/orders/N`; "Order #N placed!"; total ฿180.00; `/orders` lists it as PLACED
TEST-F03-20	AC-F03-11	E2E	alice logged in; Latte in cart; Latte set sold out in DB before submit	Place order	error message shown naming Latte; cart still has Latte; line marked

Tests are written first and must fail before the code exists (`[red]` commit, output saved in `docs/evidence/tdd/`), then pass (`[green]` commit). Test names start with the test ID.

8. Definition of Done

Shared checklist: `definition-of-done.md`. F-03 extra evidence: red then green output for every BR test (TEST-F03-02..11), E2E run for TEST-F03-19..20, screenshots of cart, rejected order, My orders and order detail.

9. Version history

Version	Date	Change	By
1	6 Nov 2026	First version, before implementation	ninja
1.1	6 Nov 2026	Shared order shape with F-04 agreed in chat: list is a plain array; malformed id → 400 (was 404). Still before implementation	ninja

Agreed with F-04 (chat, 6 Nov): F-04 adds `orders.accepted_at` (nullable) and `acceptedAt` to the shared order JSON (`null` until accepted). Whichever of F-03 / F-04 merges second adds `acceptedAt` to the order response. S4 shows the accepted time when `acceptedAt` is not null.

F-04 Operator queue + accept order — engineering contract

Sprint 1. Issue #5. Owner: A (vieng). Reviewer: B (ninja). Branch `feature/F-04-accept`. Definition of Done: `definition-of-done.md`. Status: **Specified** (committed before any F-04 code).

1. Feature and requirements

Linked	Text (short)
FR-09	Operator queue of PLACED orders, oldest first
FR-10	Accept an order: PLACED → ACCEPTED
BR-06	Only a PLACED order can be accepted, else 409
BR-07	Two operators accept at once → exactly one 200, the other 409
NFR-04	Safe concurrent accept (20 parallel accepts → exactly 1 success)
FR-04, NFR-03	OPERATOR only; shared error format

Depends on: Issue #1 (schema, seed), F-01 (`requireRole("OPERATOR")`). Orders are created by F-03; F-04 tests insert orders directly in the test database, so F-04 does not wait for F-03 code.

2. Behaviour

Workflow

1. Operator opens `/operator/orders` → client calls `GET /api/operator/orders?status=PLACED`.
2. Server returns every PLACED order of every customer (one shop), oldest `createdAt` first (ties by id).
3. Operator presses **Accept order** → `POST /api/operator/orders/:id/accept`.
4. Server checks `canTransition(current, ACCEPTED)` through one conditional update: `UPDATE orders SET status = 'ACCEPTED', accepted_at = now() WHERE id = :id AND status IN (statuses that may move to ACCEPTED)`. 1 row updated → 200 with the order. 0 rows → read the order: missing → 404; otherwise → 409 (status unchanged).
5. Client shows "Order #N accepted." and removes the card; the customer sees ACCEPTED on their order.

Order status transitions

One table in `server/src/orders/transitions.ts`, used by every status change now and later:

From	Allowed to (Sprint 1)	Added later
PLACED	ACCEPTED	REJECTED, CANCELLED (F-06)
ACCEPTED	—	PREPARING (F-07)
PREPARING	—	READY (F-07)
READY	—	PICKED_UP (F-07)
PICKED_UP, REJECTED, CANCELLED	— (final)	—

`canTransition(from, to)` returns true only for a pair in the table.

Rules, validation, boundaries

- `status` query: only `PLACED` is supported in Sprint 1; missing means `PLACED`; any other value → 400.
- `:id` must be a positive whole number, else 400; a well-formed id with no order → 404.
- 409 message: "Already accepted by another staff member." when the order is ACCEPTED; otherwise "Only PLACED orders can be accepted." `details: { status }` gives the current status.
- The race in BR-07 is decided by the database: the conditional update is atomic, so only one request can change the row.

Permissions

Endpoint	Anonymous	CUSTOMER	OPERATOR
`GET /api/operator/orders`	401	403	✓ all customers' orders
`POST /api/operator/orders/:id/accept`	401	403	✓ any order (rules above)

3. UI — S5 Operator order queue (`/operator/orders`)

Per `../../ui/ui-spec.md` S5.

- Heading "New orders" with the count; **Refresh queue** button (no auto-polling).
- Phone: one card per order — `#id`, customer name, placed time, items summary ("Latte ×2, Croissant ×1"), total `฿xx.xx`, **Accept order**. Desktop: four number cards (Waiting orders, Oldest waiting in minutes, Items to make, Queue value — worked out on the client from the list) and the orders as a table.
- States: loading placeholders; empty "No new orders. New orders appear here when you refresh."; load error "Couldn't load orders." + **Reload queue**; accepting: only that button shows "Accepting…" (disabled); success "Order #N accepted." and the card leaves; 409 "Order #N: Already accepted by another staff member." then reload; 404 "Order #N no longer exists." then reload.

4. API

Order JSON (shared with F-03): `{ id, status, total, createdAt, acceptedAt, lines: [{ menuItemId, name, unitPrice, quantity, lineTotal }] }`; money in satang, times ISO 8601. Queue items add `customer: { id, name }`. Lists are plain arrays.

Method + path	Request	Success	Errors
`GET /api/operator/orders?status=PLACED`	cookie `sid` (OPERATOR)	200 `[ { id, status: "PLACED", total, createdAt, acceptedAt: null, customer: { id, name }, lines: [...] } ]` oldest first	400 VALIDATION_ERROR (other status) · 401 · 403
`POST /api/operator/orders/:id/accept`	cookie `sid` (OPERATOR), no body	200 order with `status: "ACCEPTED"`, `acceptedAt` set	400 (bad id) · 401 · 403 · 404 NOT_FOUND "Order not found." · 409 INVALID_STATE

5. Data

Migration `add_order_accepted_at`: `orders.accepted_at TIMESTAMP NULL`, set when accepted. No other change. The existing index `(status, created_at)` serves the queue query.

6. Acceptance criteria

ID	Given / when	Then
AC-F04-01	Orders in PLACED (created at different times) and ACCEPTED; operator GETs the queue	Only PLACED orders, oldest first; each has customer name, lines (name, quantity), total
AC-F04-02	PLACED orders from alice and bob	Both customers' orders are in the queue
AC-F04-03	`?status=ACCEPTED` or `?status=nonsense`	400 VALIDATION_ERROR
AC-F04-04	A PLACED order; operator accepts it	200 with `status: "ACCEPTED"` and `acceptedAt` set; the database row is ACCEPTED; the order is no longer in the queue
AC-F04-05	An ACCEPTED order; operator accepts it again (BR-06)	409 INVALID_STATE "Already accepted by another staff member."; row unchanged

ID	Given / when	Then
AC-F04-06	An order in CANCELLED (set directly in the database); operator accepts (BR-06)	409 INVALID_STATE "Only PLACED orders can be accepted."; row unchanged
AC-F04-07	One PLACED order; 20 accept requests sent at the same time (BR-07, NFR-04)	Exactly one 200 and nineteen 409; order ACCEPTED once
AC-F04-08	Accept id 999999 (no such order); accept id `abc`	404 NOT_FOUND; 400 VALIDATION_ERROR
AC-F04-09	Customer session; no session — on both endpoints	403 FORBIDDEN; 401 UNAUTHENTICATED; order unchanged
AC-F04-10	`canTransition`	true only for PLACED → ACCEPTED; false for ACCEPTED → ACCEPTED, PLACED → READY, CANCELLED → ACCEPTED and every other pair
AC-F04-11	UI: operator logs in with 2 PLACED orders	Two cards, oldest first, heading count 2; with none: the empty message
AC-F04-12	UI: operator presses Accept order	Button shows "Accepting…"; then "Order #N accepted." and the card is gone
AC-F04-13	UI: order accepted elsewhere after the page loaded; operator presses Accept order	"Order #N: Already accepted by another staff member." and the list reloads without it

7. Test specification

Levels as in F-01 (unit / API with real Postgres / E2E). The test ID is at the start of each test name.

Test ID	AC	Level	Precondition / input	Action	Expected
TEST-F04-01	AC-F04-01	API	Seed; 3 orders: alice PLACED t1, bob PLACED t0 (older), alice ACCEPTED	GET queue as staff	2 orders `[bob t0, alice t1]`; customer names; lines; totals
TEST-F04-02	AC-F04-02	API	Same as 01	GET queue	customer ids include alice and bob
TEST-F04-03	AC-F04-03	API	Seed	GET `?status=ACCEPTED`, `?status=nonsense`	400 both
TEST-F04-04	AC-F04-04	API	One PLACED order	POST accept as staff; GET queue	200 ACCEPTED + acceptedAt; DB row ACCEPTED; queue empty
TEST-F04-05	AC-F04-05	API	One ACCEPTED order	POST accept	409 message as specified; DB row still ACCEPTED, acceptedAt unchanged
TEST-F04-06	AC-F04-06	API	One CANCELLED order	POST accept	409; DB row still CANCELLED
TEST-F04-07	AC-F04-07	API	One PLACED order	20 POST accept in parallel (`Promise.all`)	statuses: one 200, nineteen 409; DB ACCEPTED

Test ID	AC	Level	Precondition / input	Action	Expected
TEST-F04-08	AC-F04-08	API	Seed	POST accept 999999; POST accept `abc`	404; 400
TEST-F04-09	AC-F04-09	API	One PLACED order	Both endpoints as alice and anonymous	403 / 401; order still PLACED
TEST-F04-10	AC-F04-10	unit	—	`canTransition` for every (from, to) pair of the 7 statuses	true only for PLACED → ACCEPTED
TEST-F04-11	AC-F04-11	E2E	Seed; alice places 2 orders through the API	Log in as staff	2 cards oldest first, count 2
TEST-F04-12	AC-F04-12	E2E	As 11	Press Accept order on the first card	"Order #N accepted."; 1 card left
TEST-F04-13	AC-F04-13	E2E	As 11; then accept order N through the API	Press Accept order on N	409 message; N not in the list

TDD: TEST-F04-01..10 are committed failing first (`[red]`, `docs/evidence/tdd/F-04-red.txt`), then the code (`[green]`, `F-04-green.txt`). E2E tests run in Issue #10.

8. Definition of Done

See `definition-of-done.md`. F-04 specific: TEST-F04-07 runs in every test run (not skipped) because it is the evidence for BR-07.

9. Version history

Date	Version	Change	Why
6 Nov 2026	1	First contract; adds `accepted_at`	Before implementation (Issue #5); accepted time requested for the order detail timeline (chat with ninja, 6 Nov)

Product Definition of Done — Sprint 1

Applies to every Sprint 1 feature (F-01..F-04) and to the Sprint 1 release. A feature is **Done** only when every line below is true **and shown by evidence** (a test run, a commit, a PR, a screenshot) — not because someone (or an AI agent) says so.

Feature is Done when

#	Check	Evidence
1	Contract (this folder) was committed **before** the feature code, and any later change is in its version history with a reason	Git history: contract commit date < first code commit
2	Every behaviour in the contract is implemented (API, data, UI), or the gap is listed as known incomplete work	Contract vs code review in the PR
3	Every acceptance criterion has at least one planned test in the contract's test specification	Test spec table
4	Rule work used TDD: the test failed first, then passed after the code	`[red]` / `[green]` commits + `docs/evidence/tdd/<feature>-red.txt` and `-green.txt`
5	All automated tests pass (`npm test`; E2E for the feature when its screens exist); none are skipped, disabled or weakened to pass	Test output in the PR description
6	Access is enforced on the server: 401 without a session, 403 for the wrong role, 404 for another user's record	API tests
7	Errors use `{ error: { code, message, details } }`; no stack traces or secrets in responses, logs or Git	API tests + review
8	Code builds and type-checks (`npm run build`)	Build output
9	A PR into `sprint-1` with "Closes #n", linked to its Issue (branch + PR in "Development"), reviewed and **approved by the partner** with a specific comment, merged with a merge commit by the partner	GitHub PR page
10	README, `.env.example` and docs are up to date with any new command, variable or decision	PR diff
11	Board card moved through every column to Done; Issue closed with a comment	Project board

Sprint 1 release is Done when

#	Check	Evidence
R1	F-01..F-04 and Docs A–D are Done (above)	Board, closed Issues
R2	On `sprint-1`: fresh install → migrate → seed → `npm test` and `npm run test:e2e` pass; build passes	Saved output with the tested commit id
R3	Traceability: every Sprint 1 AC maps to a test ID, test file/name and result	`docs/traceability.md`
R4	Both students rechecked everything on `sprint-1` and agree it is done	Chat / PR comment from both
R5	vieng decides the merge into `main` and the tag `v0.1.0-sprint1` (AIs do not do this on their own)	Release PR + tag
R6	Known incomplete work and any failed or skipped test are written down	Part F of the PDF

Version history

Date	Version	Change	Why
6 Nov 2026	1	First version	Sprint 1 contracts

Part F — Working Sprint 1 and verification

docs/evidence/part-f.md is not in this commit yet — this section is generated from it when it is added.